

M1 – MASTER SAR
Premier examen réparti
Novembre 2017

2 heures – Tout document papier et calculatrice autorisés
Barème donné à titre indicatif

L. Arantes, P. Cadinot, S. Dubois, J. Lejeune, J. Sopena, P. Sens

I. SYNCHRONISATION (6 POINTS)

On souhaite offrir un mécanisme de synchronisation entre des processus frères (i.e., partageant le même père). La fonction `int susp_sibling()` bloque l'appelant à la priorité PSIBL tant qu'au moins un de ses frères n'a pas appelé `void signal_sibling()`.

`susp_sibling()` retourne 0 à son réveil. En revanche, `susp_sibling` retourne 1 et ne se bloque pas si l'appelant n'a pas de père ou si `signal_sibling()` a déjà été appelée.

I.1. (2,5 points)

Dans un premier temps on fait l'hypothèse que lors de l'appel à `susp_sibling`, il existe toujours un frère à l'appelant.

Donnez le code de **l'appel système** `sys_susp_sibling()` appelé par la fonction `int susp_sibling()`.

Vous ne pouvez pas ajouter de champs supplémentaires dans la structure `proc`. En revanche, si nécessaire vous pouvez définir de nouveaux flag à mettre dans le champ `p_flag` de la structure `proc`.

```
sys_susp_sibling() {
{
    register struct proc *p;

    if (u.u_procp->p_ppid == 1) /* Pas de père */
        goto endf;

    for (p = &proc[0]; p < &proc[NPROC]; p++)
        if (p->p_pid == u.u_procp->p_ppid) { /* Chercher le père */
            if (p->p_flag & SSYNCH) /* Déjà appelé */
                goto endf;
            sleep(p+1, PSIB)
            u.u_ar0[R0] = 0;
            return;
        }
    }
endf :
```

```

u.u_ar0[R0] = 1;
return;
}

```

I.2. (1,5 points)

Donnez le code de l'**appel système** `sys_signal_sibling()` appelé par la fonction `void signal_sibling()`.

```

sys_signal_sibling() {
{
    register struct proc *p;
    for (p = &proc[0]; p < &proc[NPROC]; p++)
        if (p->p_pid == u.u_procp -> p_ppid) {
            p->p_flag |= SSYNCH;
            wakeup(p+1);
            return;
        }
}
}

```

I.3. (2 points)

On suppose maintenant que `susp_sibling` ne doit pas se bloquer et retourner 1, si au moment de l'appel aucun frère n'existe.

Modifiez le code de l'**appel système** `sys_susp_sibling()` en conséquence

```

sys_susp_sibling() {
{
    register struct proc *p;
    int sib = 0;

    if (u.u_procp == 1) /* Pas de père */
        goto endf ;
    for (p = &proc[0]; p < &proc[NPROC]; p++)
        if (p->p_ppid == u.u_procp -> p_ppid && p->p_pid != u.u_procp ->
p_pid) {
            sib = 1 ; /* Il existe un frere */
            break ;
        }
    }
    if ( !sib) /* Pas de frère */
        goto endf ;
    for (p = &proc[0]; p < &proc[NPROC]; p++)
        if (p->p_pid == u.u_procp -> p_ppid) { /* Chercher le père */
            if (p->p_flag & SSYNCH) /* Déjà appelé */
                goto endf;
            sleep(p+1, PSIB)
            u.u_ar0[R0] = 0;
            return;

```

```

    }
}
endif :
    u.u_ar0[R0] = 1;
    return;
}

```

II. TEMPS (6 POINTS)

Nous souhaitons pouvoir offrir la fonction utilisateur suivante

```
int at(int nbsec, void (*f)()).
```

La fonction `at` permet de planifier l'exécution d'une fonction utilisateur sans argument (désignée par le pointeur `f`) dans `nbsec` secondes. La fonction devra renvoyer 0 si l'enregistrement de la planification s'est bien faite, -1 sinon.

II.1. (1 point)

Une première solution purement utilisateur qui éviterait de modifier le système serait la redéfinition du handler du `SIGALRM`. Codez cette première version de `at()` grâce aux fonctions utilisateurs `alarm()` et `signal()` vues en TD.

```

int at(int nbsec, void (*f)()){
    signal(SIGALRM,f);
    alarm(nbsec);
    return 0;
}

```

On veut offrir la possibilité de planifier plusieurs fonctions, c'est à dire qu'un appel à `at` n'annule pas la ou les planification(s) précédente(s) qui ne sont pas encore déclenchée(s).

II.2. (0,5 point)

Expliquez pourquoi la solution de la question précédente ne respecte pas cette spécification.

On ne peut que planifier un seul `SIGALRM` à la fois. en effet un appel à la fonction `alarm` écrase le champs `p_clktim`, on perd ainsi les planifications précédentes.

Notre deuxième solution serait de programmer un appel système `sys_at()` qui implante dans le noyau la fonction `at()`. L'idée serait d'utiliser le mécanisme des timeout qu'offre le noyau.

II.3. (0,5 point)

Expliquez pourquoi il ne faut pas stocker directement le paramètre f dans le champ c_func de la structure callo.

Le pointeur de fonction de callo sera du code qui s'exécutera en mode système, hors la fonction f est une fonction utilisateur. On offrirait donc la possibilité d'exécuter du code U en mode S ce qui serait une faille de sécurité.

II.4. (1 point)

Quelle fonction faudrait-il stocker dans le champ c_func ? En déduire ce que vous stockerez dans le champs c_arg.

On pourrait directement faire appel à une fonction sendsig sans le num qui permet de modifier la pile U depuis le mode S . Dans c_arg on stocke le pointeur de fonction f.

II.5. (3 points)

Codez l'appel système sys_at(). Celui-ci renverra -1 si la table des callout est pleine et mettra dans ce cas le flag FULL au champ u_error.

```
sys_at(){
    int t = u.u_arg[0]; //recupération du temps en secondes
    void (*f)() = u.u_arg[1]; //récupération du pointeur de fonction
    struct callo* p;

    ps = gpl()
    spl(CLINHB)
    p = &v.ve_callout[0];
    while(p->c_func !=0)
        p++;
    if(p == &v.ve_callout[v.v_callout - 2]){
        u_ar0[RO]=-1;
        u.u_error=FULL;
    }
    else{
        timeout(sendsig, f ,t*HZ);
        u_ar0[RO]=0;
    }
    spl(ps)
}
```

III. ORDONNANCEMENT (4 POINTS)

Dans cet exercice on se propose d'implémenter un nouvel appel système permettant de laisser sa place en forçant une commutation pendant au moins « n » élections consécutives s'il y a au moins un autre processus candidat.

III.1. (0,5 point)

Pour réaliser cet appel système il va être nécessaire de modifier une des deux structures caractérisant les processus. Quelle est cette structure ? Justifiez votre choix.

Il faut modifier la structure proc (et non la structure user) car les informations ajoutées vont être utilisées par le swaper lors de l'élection (dans la fonction swtch()).

III.2. (3,5 points)

Donnez le code complet d'un tel appel système (sys_fair()) ainsi que les modifications à faire dans la fonction swtch(). Vous pourrez alors utiliser les numéros de lignes pour éviter d'en recopier tout le code.

Votre appel système devra retourner une erreur si le nombre d'élections à « sauter » n'est pas strictement positif.

Dans le fichier proc.h, ajout dans la structure proc d'un champ : int fair ;

Code de l'appel système

```
sys_fair()
{
    if (u.u_arg[0] > 0) {
        u.u_procp->fair = u.u_arg[0];
        runrun++ ;
    } else {
        u.u_error = EVAL ;
    }
}
```

Dans le fichier swtch.c :

```
7 swtch()
8 {      at_least_one_fair = 0 ;
...
36 ...
    if (rp->fair) {
        rp->fair-- ;
        at_least_one_fair = 1 ;
    }
37 ...
38 if (rp->fair==0 && rp->p_pri < n) {
39 ...

47 ...
    if (at_least_one_fair) goto loop ;
48 ...
... .
```

IV. SIGNAUX - APPEL SYSTEM SIGNAL (4 POINTS)

Nous voulons que l'appel système `int signal(int signo, void *handler (void))` permette au programmeur d'indiquer plusieurs handlers à exécuter lors de la délivrance du signal `signo`. Pour cela, le deuxième paramètre de la fonction `signal` est un vecteur où chaque entrée correspondra à une fonction (handler) à être exécutée, lors de la délivrance du signal. Les fonctions doivent être exécutées dans l'ordre croissant des entrées du vecteur dont la dernière entrée est égale à `NULL`. Il peut y avoir un maximum de `NB_HANDLER`.

Exemple du nouveau appel à `signal(int signo, void *handler (void))` :

```
void handler1( ){
    ....
}

void handler2( ){
    ....
}

int main (int argc, char** argv) {

    void* handlers[3];
    handlers[0]=handler1;
    handlers[1]=handler2;
    handler[2]=NULL;
    .....
    signal (SIGUSR1, handlers);
    ...
}
```

Dans cet exemple, lorsqu'un signal `SIGUSR1` est délivré au programme, les fonctions `handler1 ()` et `handler2 ()` seront exécutées dans cet ordre.

Pour ce changement, le champ `u_signal` de la structure `struct user` et la fonction du noyau `ssig ()`, appelée par `signal ()`, ont été aussi modifiées de la façon suivante:

```
struct t_u_signal{
    int handlers[NB_HANDLER+1];
}

struct user {
    ....
    struct t_u_signal u_signal[NSIG];
}

ssig ( ) {
    register a=u.u_arg[0];
    int i=0;
    ....
```

```

while ((u.u_arg[1][i]!=NULL) && (i <=NB_HANDLER)) {
    u.u_signal[a].handlers[i]=u.u_arg[1][i];
    i++;
}
u.u_signal[a].handlers[i] = NULL;
}

```

En reprenant l'exemple précédent, on aurait après l'appel à `signal ( )` :

```

u.u_signal[SIGUSR1].handlers [0] =handler1;
u.u_signal[SIGUSR1].handlers [1] =handler2;
u.u_signal[SIGUSR1].handlers [2] =NULL;

```

IV.1. (4 points)

Modifiez les fonctions du noyau `psig ( )` (plutôt les lignes 108 à 113) et `sendsig (void*handler,int num)`, vues en TDs pour que tous les handlers définis lors de l'appel de la fonction `int signal(int signo, void *handler (void) )` soient exécutés dans l'ordre lorsque le signal `signo` est délivré. Vous pouvez ajouter des variables locales dans les fonctions et changer le prototype de la fonction `sendsig`.

```

psig ( ) {

    register n ;
    int i=0;
    register struct proc * rp ;
    void* handlers[NB_HANDLER];

    rp=u.u_procp;

    n=fsig(rp);
    if(n==0)
        return ;
    rp->p_sig &= ~(1<<(n- 1) ) ;

    /* lignes 108 à 113 */
    if (u.u_signal[n].handlers[0] != NULL) {
        while ((u.u_signal[n].handlers[i] != NULL) && (i <
NB_HANDLER)) {
            handlers[i]=u.u_signal[n].handlers[i];
            i++;
        }

        u.u_error = 0 ;
        u.u_signal[n].handler[0]=NULL;
        sendsig (handlers, i);
        return;
    }
}

```

```

switch (n) {
    ...
}
exit (n);
}

/* dans le TD, le deuxième paramètre n'est pas utilisé par
sendisg*/

sendsig (void* handlers, int num ) {
    int nb = num;

    while (nb > 0) {
        sp = u.u_ar0[SP]-4;
        grow(sp) ;
        u.u_ar0[SP] = sp ;
        if (nb == num)
            suword (sp,u.u_uar0[PC]); /*sp [0] = PC */
        else
            suword (sp, handlers[nb]);; /*sp [0] = handlers[nb] */
        nb --;
    }
    u.u_ar0 [PC] = handler[0] ;
}

```

Autre solution possible (sans modification de sendsig):

```

psig ( ) {

    register n ;
    int i=0;
    register struct proc * rp ;
    int nh = 0;

    rp=u.u_procp;

    n=fsig(rp);
    if(n==0)
        return ;
    rp->p_sig &= ~(1<<(n- 1) ) ;

    /* lignes 108 à 113 */
    if (u.u_signal[n].handlers[0] != NULL) {
        while ((u.u_signal[n].handlers[i] != NULL) && (nh <
NB_HANDLER))
            nh++;

        u.u_error = 0 ;
        u.u_signal[n].handler[0]=NULL;
        while (i--)

```

```
        sendsig (u.u_signal[n].handlers[i], n);
    return;
}
switch (n) {
    ...
}
exit (n);
}
```

ANNEXE

```
1  /*
2  * This routine is called to reschedule the CPU.
3  * if the calling process is not in RUN state,
4  * arrangements for it to restart must have
5  * been made elsewhere, usually by calling via sleep.
6  */
7  swtch()
8  {
9  static struct proc *p;
10 register i, n;
11 register struct proc *rp;
12
13 if(p == NULL)
14     p = &proc[0];
15 /*
16  * Remember stack of caller
17  */
18 savu(u.u_rsav);
19 /*
20  * Switch to scheduler's stack
21  */
22 retu(proc[0].p_addr);
23
24 loop:
25 runrun = 0;
26 rp = p;
27 p = NULL;
28 n = 128;
29 /*
30  * Search for highest-priority runnable process
31  */
32 i = NPROC;
33 do {
34     rp++;
35     if(rp >= &proc[NPROC])
36         rp = &proc[0];
37     if(rp->p_stat==SRUN && (rp->p_flag&SLOAD)!=0) {
38         if(rp->p_pri < n) {
39             p = rp;
40             n = rp->p_pri;
41         }
42     }
43 } while(--i);
44 /*
45  * If no process is runnable, idle.
46  */
47 if(p == NULL) {
48     p = rp;
49     idle();
50     goto loop;
```

```

51  }
52  rp = p;
53  /*
54   * Switch to stack of the new process and set up
55   * his segmentation registers.
56   */
57  retu(rp->p_addr);
58  sureg();
59  /*
60   * If the new process paused because it was
61   * swapped out, set the stack level to the last call
62   * to savu(u_ssav). This means that the return
63   * which is executed immediately after the call to aretu
64   * actually returns from the last routine which did
65   * the savu.
66   *
67   * You are not expected to understand this.
68   */
69  if(rp->p_flag&SSWAP) {
70      rp->p_flag =& ~SSWAP;
71      aretu(u.u_ssav);
72  }
73  /*
74   * The value returned here has many subtle implications.
75   * See the newproc comments.
76   */
77  return(1);
78  }

```

```

91 /*
92  * Perform the action specified by
93  * the current signal.
94  * The usual sequence is:
95  *if(issig())
96  *psig();
97  */
98 psig()
99 {
100     register n, p;
101     register struct proc *rp;
102
103     rp = u.u_procp;
104     n = fsig(rp);
105     if (n==0)
106         return;
107     rp->p_sig &= ~(1<<(n-1));
108     if((p=u.u_signal[n]) != 0) {
109         u.u_error = 0;
110         u.u_signal[n] = 0;
111         sendsig(p, n);
112         return;
113     }
114     switch(n) {
115
116         case SIGQUIT:
117         case SIGINS:
118         case SIGTRC:
119         case SIGIOT:
120         case SIGEMT:
121         case SIGFPT:
122         case SIGBUS:
123         case SIGSEG:
124         case SIGSYS:
125             if(core())
126                 n += 0200;
127     }
128     exit(n);
129 }
...
150 sendsig(void *handler, int num) {
151     sp = u.u_ar0[SP] - 4;
152     grow(sp);
153     u.u_ar0[SP] = sp;
154     suword(sp, u.u_ar0[PC]); /* sp[0] = PC */
155     u.u_ar0[PC] = handler;
156 }

```